

CSC490: Project Report

Team Members

Name	Student ID
Aarya Prakash	1008790257
Derek Huynh	1008849052
Merrick Liu	1008951920
Rhys Balevicius	1000801974

University of Toronto
Department of Computer Science

1 Abstract

AI coding agents have dramatically accelerated software development, yet they remain fundamentally limited by context: they can only reason over the files immediately visible in their working session. In large-scale software systems—spanning dozens of repositories, layered service boundaries, and years of accumulated architectural conventions—this constraint is a critical bottleneck. Agents that lack knowledge of upstream API contracts, cross-repository dependencies, or established design patterns produce incorrect, inconsistent, or redundant code.

We present **Lighthouse**, a search engine designed specifically to address this gap for AI coding agents. Lighthouse indexes GitHub repositories through durable, Temporal-orchestrated pipelines that produce semantically chunked, embedded representations of source code and auto-generated documentation. At query time, a hybrid retrieval strategy combining dense vector search (Milvus), full-text search (PostgreSQL), LLM-driven keyword expansion (OpenAI), and reranking (Cohere) to surface the most relevant context from across all indexed repositories. This context is delivered to agents via the Model Context Protocol (MCP), a standardized interface that requires no changes to the agent or its tooling.

2 Introduction and Problem Statement

AI coding agents—tools such as GitHub Copilot, Cursor, and Claude Code—have become central to modern software development. They automate routine implementation, synthesize boilerplate, and increasingly attempt to resolve multi-step engineering tasks with minimal human guidance. Yet despite rapid capability gains, these agents share a structural limitation: their context window is bounded by the files open in the current session. For self-contained tasks within a single file, this is often sufficient. For the kinds of tasks that arise in real engineering organizations—tasks that cross service boundaries, depend on the implicit contracts of internal libraries, or require knowledge of established architectural patterns—it is not.

3 Introduction and Problem Statement

3.1 The Problem

AI coding agents—tools such as GitHub Copilot, Cursor, and Claude Code—have become central to modern software development, but they remain constrained by the context available in a single session. That limitation is manageable for isolated edits, but it breaks down for real-world engineering tasks that span repositories, depend on internal service contracts, or rely on organization-specific architectural patterns.

As a result, agents often make plausible but incorrect changes: they call nonexistent or outdated endpoints, omit required conventions such as authentication or wiring patterns,

duplicate infrastructure that already exists, or miss downstream dependencies during refactors. The issue is not weak reasoning, but missing context.

3.2 Why Existing Tools Fall Short

Existing tools solve adjacent problems, but not agent-native context retrieval across an organization’s codebase. IDE autocomplete tools improve local productivity, but they do not index non-local repositories or expose retrieved context programmatically to agents. Code search platforms are built for humans, not for agents that need structured context during active reasoning, while general-purpose RAG frameworks typically require custom setup and lack code-aware, continuously updated indexing. It is worth noting that much of the industry is realizing this is a problem and are moving toward MCP and indexing multiple repositories for context.

3.3 Lighthouse: A Context Layer for Coding Agents

We propose and implement **Lighthouse**, a retrieval layer for AI coding agents. At a high level, Lighthouse consists of:

1. A **production-grade ingestion pipeline** with full and incremental indexing of GitHub repositories, durable execution via Temporal, and a publish-and-switch model that eliminates search gaps during re-indexing.
2. A **multi-stage federated hybrid retrieval strategy** combining dense vector search, sparse full-text search, LLM keyword extraction, and reranking from multiple sources (wiki and code).
3. An **automated wiki generation system** that uses an LLM to synthesize repository documentation from indexed source code, expanding the retrievable surface beyond raw code.
4. An **agent-native MCP interface** that exposes code and wiki search as first-class tools consumable by AI coding agents with no custom integration.
5. A **reproducible evaluation framework** integrating synthetic API-contract benchmarks to measure the impact of Lighthouse on agent task completion.

4 Related Work

Lighthouse sits at the intersection of code retrieval, retrieval-augmented generation, and AI agent tooling. We survey the most relevant prior work across two dimensions: industry products and academic literature.

4.1 Industry Solutions

Several commercial products target developer productivity through code search or AI-assisted coding, but none provide a general-purpose, agent-native context retrieval layer. Table 1 summarizes the most relevant products and their key limitations relative to Lighthouse.

Table 1: Comparison of industry tools to Lighthouse.

Tool	Approach	Limitation vs. Lighthouse
Cursor, Copilot, Codex, etc.	The standard coding agents, tab completion, chat, tool calls, etc.	No native cross-repository indexing.
Sourcegraph	Enterprise code search platform.	Most similar to Lighthouse. However, only surfaces and indexes code.
AugmentCode	Codebase-aware AI coding assistant that indexes large monorepos for context.	Tightly coupled to a proprietary IDE plugin; no open agent interface.
Glean	Enterprise knowledge retrieval across code, docs, and tickets.	Not code-specific; does not perform semantic code chunking or vector search.
Cognition Deep Wiki	Automatically generates natural-language wiki documentation for a repository using an LLM agent.	Documentation-generation only; no retrieval interface, no MCP integration, no code search.
grep.app	Fast regex and literal search over public GitHub repositories via a web interface.	Lexical search only; no semantic/vector retrieval, no agent interface, no private repo support.

4.2 Academic Work

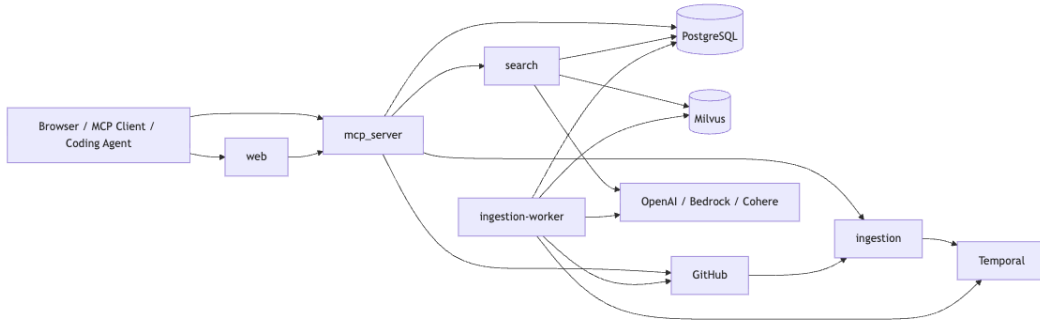
Lighthouse builds on several foundational techniques from the information retrieval and NLP literature. Table 2 summarizes the key works most directly relevant to our design.

Table 2: Academic works that inform the Lighthouse design.

Work	Contribution	Relevance to Lighthouse
RAG [1]	Augments LLM generation with non-parametric retrieval over a dense document index.	Establishes the retrieval-then-generate paradigm that Lighthouse operationalizes for code.
DPR [2]	Dense bi-encoder retrieval using dot-product similarity over fixed-dimensional vectors.	Basis for our Milvus-backed vector search stage using OpenAI <code>text-embedding-3-large</code> .
CodeSearchNet [3]	Large-scale code-docstring pairs across six languages for training code search models.	Provides a reference approach to NL→code retrieval.
Reciprocal Rank Fusion [4]	Score-free fusion of ranked lists by inverse rank position.	Directly implemented in our hybrid search pipeline to combine vector and full-text result lists.
CodeRAG-Bench [5]	Benchmark for RAG-augmented code generation covering eight task types and five retrieval corpora.	Inspiration for our evaluation designs; establishes retrieval+generation as a joint evaluation target.

5 Technical Approach, Solution, and Designs

Lighthouse is composed of four microservices—*Ingestion*, *Search*, *MCP Server*, and *Web*—backed by a dual-store database layer (PostgreSQL for relational metadata and full-text search; Milvus for dense vector retrieval) and a durable workflow engine (Temporal) for long-running indexing jobs. Figure 1 shows the high-level topology.

**Figure 1:** Lighthouse system architecture. Coding agents interact exclusively through the MCP Server. Milvus and PostgreSQL form the dual-store retrieval backend.

5.1 Ingestion Pipeline

The ingestion pipeline converts raw GitHub repositories into a semantically indexed, version-aware corpus of code chunks and wiki documentation. It is implemented as a set

of Temporal workflows executed by a dedicated worker process.

5.1.1 Code Chunking

Every file in a repository passes through a chunker before embedding. Lighthouse supports two strategies, selected at deployment time via the `CHUNKER_STRATEGY` environment variable:

- **AST-aware chunking** (`ast_code`): Parses each source file into a syntax tree and splits at top-level declaration boundaries (functions, classes, methods). Chunks are capped at `CHUNK_MAX_LINES= 50` lines with a `CHUNK_OVERLAP_LINES= 10` line overlap to preserve context across boundaries. This is the default strategy for all files whose language has a supported grammar.
- **Sliding window** (`sliding_window`): Falls back to fixed-size line windows with the same parameters for languages without AST support or for files that fail parsing.

Both strategies skip lock files, binary files, and files exceeding 1 MB. Language detection is performed per file using extension heuristics. The result of chunking is a set of `StagingChunk` rows in PostgreSQL, each recording the file path, start and end line numbers, raw content, and a batch identifier.

5.1.2 Embedding Generation

Chunked text is embedded in parallel batches of 64 via one of two pluggable providers:

- **OpenAI text-embedding-3-large**: 3072-dimensional vectors; default for code embeddings.
- **AWS Bedrock amazon.titan-embed-text-v2:0**: 1024-dimensional vectors; used when running fully within AWS without third-party API dependencies.

The embedding provider is selected at deployment time; both dimensions are supported throughout the pipeline (Milvus collections are created with the correct dimension on first use). Embedding activities carry a retry policy of five attempts with two-second exponential backoff up to a 60-second cap to absorb transient API rate limits.

5.1.3 Staging and Publish Model

To guarantee that live search results are never corrupted by a partially completed re-indexing run, Lighthouse uses a *staging-and-publish* pattern:

1. New chunks are written to a `StagingChunk` table.
2. Embedding vectors are filled in-place in the staging table.
3. A single atomic *publish* activity promotes chunks to the `Chunk` table, generates a fresh UUID `publish_id`, and updates `IndexedFile.active_publish_id` for every affected file.
4. Vectors are then inserted into the Milvus `chunk_embeddings` collection under the new `publish_id`.

5. Old `publish_id` entries are deleted from Milvus and PostgreSQL asynchronously in a best-effort cleanup activity.

At query time, the search service filters Milvus results to match the current `active_publish_id` per file, ensuring stale vectors are invisible even before the asynchronous cleanup completes. If a workflow fails at any point before the publish step, staging rows are discarded and live data is untouched.

5.1.4 Temporal Workflow Orchestration

All multi-step indexing work is expressed as Temporal workflows, providing durable execution, automatic retries at the activity level, and visibility into in-flight jobs. Three workflows are defined:

- **IndexBranchWorkflow:** Full branch indexing. Executes the sequence: *clone* → *chunk* → *embed (parallel batches)* → *publish* → *GenerateWikiWorkflow* → *cleanup*.
- **IncrementalIndexWorkflow:** Push-event triggered. Receives a GitHub push webhook, coalesces additional pushes that arrive within a 15-second window via Temporal signals, computes the git diff between **before** and **after** commits, and runs the embedding pipeline only on changed files. Unchanged files retain their existing vectors.
- **GenerateWikiWorkflow:** Child workflow spawned by either indexing workflow (best-effort; failure does not fail the parent). Synthesizes natural-language wiki documentation from the indexed code (Section 5.3).

5.2 Hybrid Retrieval Pipeline

At query time, Lighthouse executes a multi-stage retrieval pipeline that fuses dense vector search, sparse full-text search, LLM-driven query expansion, and neural reranking. Three strategies are available, selected per request via the `context_sources` parameter.

5.2.1 Dual-Store Backend

PostgreSQL stores chunk content, file paths, line-range metadata, and `publish_id` versioning. Full-text search uses PostgreSQL’s built-in `tsvector/tsquery` operators (`@@`), enabling language-aware stemming and lexeme indexing over chunk content.

Milvus stores embedding vectors in two collections: `chunk_embeddings` (code) and `wiki_embeddings` (wiki pages). Both use an HNSW index with $M = 16$, *efConstruction* = 256, and cosine similarity as the distance metric. Each Milvus record carries `repository_id`, `branch`, `file_path`, and `publish_id` scalar fields to support filtered approximate nearest-neighbor search.

The search service issues Milvus queries with a `publish_id` filter derived from the current `IndexedFile.active_publish_id`, so stale vectors introduced by a previous indexing run are excluded from results even before the cleanup activity removes them.

5.2.2 Reciprocal Rank Fusion

Vector search and full-text search return independent ranked lists. Lighthouse merges them using Reciprocal Rank Fusion (RRF) [4]:

$$\text{score}(d) = \sum_{i=1}^N \frac{1}{k + r_i(d)} \quad (1)$$

where $r_i(d)$ is the rank of document d in list i (or ∞ if absent), and $k = 60$ is a dampening constant that prevents top-ranked items from dominating the fusion score. RRF requires no learned weights and remains effective when the two lists have very different score scales, which is typical when combining cosine similarities with PostgreSQL full-text scores.

5.2.3 LLM-Combined Multi-Source Strategy

For queries that benefit from both code and wiki context, Lighthouse provides a four-phase `LLMCombinedSearchStrategy`:

1. **Phase 1 (parallel)**: An LLM (`gpt-5.4-nano`) extracts 3–5 precise keyword terms from the natural-language query. Concurrently, the query is embedded and used to retrieve candidates from both the code and wiki Milvus collections.
2. **Phase 2 (parallel)**: Four PostgreSQL FTS queries execute simultaneously: code with the original query, code with LLM keywords, wiki with the original query, and wiki with LLM keywords.
3. **Phase 3**: RRF is applied within each domain (code, wiki) to merge vector and FTS results. A cross-domain RRF then combines the two domain-level fused lists.
4. **Phase 4**: The top candidates are submitted to Cohere `rerank-v4.0-pro` for neural cross-attention reranking. If the LLM or reranking call fails, the pipeline degrades gracefully to Phase 3 RRF results.

5.3 Automated Wiki Generation

Raw code search returns syntactically precise snippets but cannot answer architectural or behavioral questions (e.g., “how does authentication flow across services?”). To address this, Lighthouse runs `GenerateWikiWorkflow` after each full indexing run:

1. **Seed**: Up to 500 file paths and 20 representative code chunks are sampled from the indexed branch.
2. **Structure generation**: The LLM is prompted with the sampled paths and chunks to produce a wiki outline: a set of top-level sections, each with titled pages and one-sentence descriptions.
3. **Page generation**: Pages are generated in parallel batches of eight, with the relevant code chunks injected as context per page.
4. **Embedding and storage**: Completed pages are embedded in batches of 512 and inserted into the Milvus `wiki_embeddings` collection. Page records (title, slug, section path, markdown content) are persisted in PostgreSQL.

The wiki generation uses the same staging-and-publish pattern as code indexing (`StagingWikiPage` → `WikiPage`). Wiki pages are searchable via `HybridWikiSearchStrategy` (vector + FTS + RRF) or as part of the full `LLMCombinedSearchStrategy`.

5.4 MCP Interface and Agent Integration

Lighthouse exposes its retrieval capabilities to coding agents via the **Model Context Protocol (MCP)**, mounted at `/mcp` on the MCP Server service using the FastMCP library. Agents connect once with a long-lived MCP token and interact through a standard tool-call interface with no Lighthouse-specific client code.

The MCP toolset includes:

- `search_code(repository, query, branch, top_k)`
- `search_wiki(repository, query, branch, top_k)`
- `list_user_repos()`
- `add_user_repo(url, branches)`
- `get_wiki(repository, branch)`

Authentication uses GitHub OAuth for the initial login flow. The server issues encrypted session tokens (Fernet, 7-day TTL) for the web dashboard and long-lived revocable MCP tokens for agent clients. Inter-service calls (MCP Server → Search, MCP Server → Ingestion) are protected by a shared 32-byte `INTERNAL_SERVICE_TOKEN` validated with constant-time comparison (`secrets.compare_digest`).

5.5 Infrastructure and Deployment

5.5.1 AWS Architecture

All five containerized workloads (web, MCP server, search, ingestion API, ingestion worker) run as ECS Fargate tasks in private subnets of a dedicated VPC (10.0.0.0/16 across two availability zones). An Application Load Balancer in the public subnet routes external traffic: the root path serves the React web UI, `/v1/*` and `/mcp/*` route to the MCP Server, and `/webhook` routes to the Ingestion service for GitHub push events. Internal service-to-service calls resolve over AWS Cloud Map private DNS (e.g., `search.lighthouse-prod.local:8002`).

Milvus runs as a standalone instance on an EC2 `t3.large` in a private subnet, with etcd and MinIO co-located via Docker Compose on the same host. This configuration avoids the operational complexity of a distributed Milvus cluster while remaining adequate for the current scale. The Milvus host is accessible to ECS tasks via a security group rule permitting port 19530 from the VPC CIDR.

PostgreSQL is provisioned as Amazon RDS (version 17.7, `db.t3.micro`, 20 GB gp3) in a private subnet DB group, accessible only from ECS task security groups.

Temporal is consumed as a managed cloud service (Temporal Cloud), eliminating the need to operate the Temporal server cluster in production.

Runtime configuration for each service is stored as an encrypted JSON `SecureString` in AWS SSM Parameter Store (e.g., `/lighthouse/prod/mcp-server`). ECS task definitions are granted `ssm:GetParameter` and `kms:Decrypt` via least-privilege IAM roles; services fetch their config at startup. This keeps secrets out of source control and environment variables while supporting per-environment overrides.

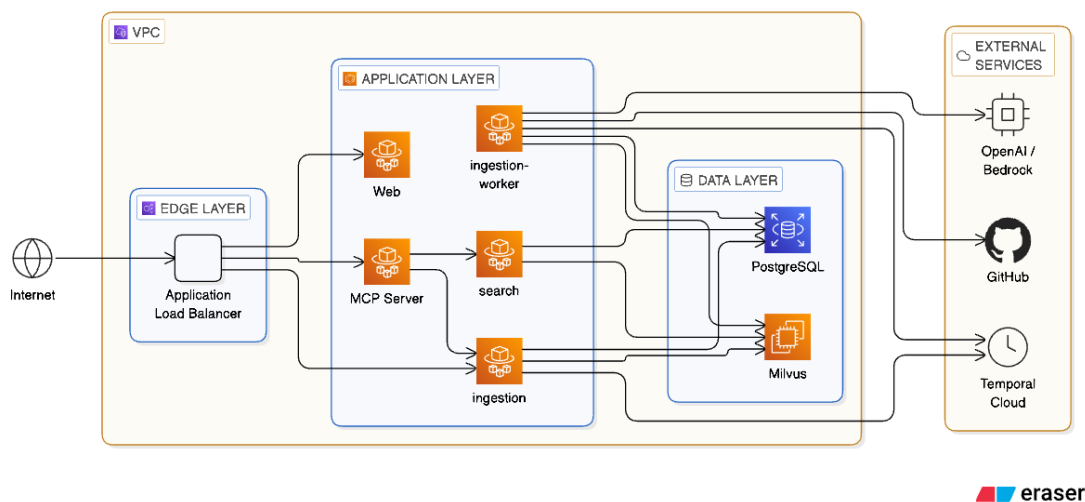


Figure 2: AWS production topology. All application services run as Fargate tasks in private subnets; Milvus is deployed as a standalone Docker Compose stack on a private EC2 instance.

5.5.2 Infrastructure as Code

The full AWS stack is defined in Terraform across four modules: **networking** (VPC, subnets, NAT gateway, VPC endpoints), **compute** (ECS cluster, task definitions, ALB, EC2 Milvus, Cloud Map), **database** (RDS subnet group and instance), and **storage** (S3 data-lake bucket). A single deployment script (`scripts/deploy_aws_stack.sh`) generates per-service secrets, writes SSM parameters, runs `terraform apply`, executes Alembic database migrations as a one-off ECS task, and forces service redeployment—providing a fully reproducible push-to-production workflow from a single command.

All Python services use multi-stage Docker builds (builder stage installs `uv` dependencies; runtime stage copies only the virtual environment) to minimize image size. Images are built and pushed to ECR by `scripts/build_and_push_images.sh` and tagged with the git commit SHA for traceability. GitHub Actions runs the full test suite (`pytest`) on every push and pull request to `main`, with line coverage reported as a PR comment.

Table 3: Synthetic task families used in the reported runs.

Family	Task type	Tasks	Role
API contract mismatch	<code>api_contract_mismatch</code>	10	Dual-repo repair tasks centered on recovering provider-library semantics.
API feature integration	<code>feature_api_integration</code>	10	Dual-repo feature tasks implemented against provider APIs.
Documentation behavior mismatch	<code>doc_behavior_mismatch</code>	10	Dual-repo repair tasks where documented behavior matters.
Documentation-guided feature work	<code>feature_doc_guided</code>	10	Dual-repo feature tasks guided by documentation.
Type coercion repair	<code>data_type_coercion</code>	10	Single-repo repair controls focused on local code understanding.
Wrong-operator repair	<code>logic_wrong_operator</code>	10	Single-repo repair controls focused on local bug localization.
Total	—	60	40 repair tasks and 20 feature tasks overall.

6 Experiments, Key Metrics, and Results

6.1 Synthetic Benchmark Dataset

We evaluate Lighthouse on a synthetic benchmark executed by the `packages/eval` pipeline. In the runs discussed here, we use six task families with 10 tasks per family, for 60 tasks total. These 10-task slices are selected deterministically by seed, and the per-family sample size is configurable through the evaluation CLI and matrix configuration.

The tasks are generated in two different ways. Repair families start from a clean repository template and introduce buggy mutants via `buggy_patch_path` diffs; the corresponding `gold_patch_path` diffs restore the intended behavior. Feature families instead define missing or stubbed functionality together with new tests that specify the desired behavior, so the model must synthesize an implementation rather than undo a single mutation. The most important family for Lighthouse is the API contract-mismatch family: it is built from a small set of consumer/provider contract motifs that are expanded into a larger bank of concrete variants. This makes it especially well aligned with our core value proposition of surfacing non-local dependency semantics.

Every task carries executable supervision and lightweight relevance labels. In addition to the buggy and gold patches, each manifest records `pytest_targets`, `expected_relevant_files`, `expected_relevant_symbols`, `visible_api_names`, and `test_context`. The evaluator validates each task by checking that the buggy state fails on the declared tests and that the gold patch applies cleanly and makes those tests pass. A model output is counted as successful only if its extracted patch applies with `git apply` and the declared tests pass.

The benchmark is also motivated by experimental-validity concerns. Existing public coding benchmarks are useful, but they create a standing risk that some benchmark instances, tests, or solution patterns were already present in model training corpora. That makes it harder to separate genuine reasoning and retrieval from benchmark memorization. By generating the benchmark instances locally from templates, mutated patches, and newly instantiated tests, we substantially reduce this contamination risk: the exact evaluation tasks and test cases used in our runs are not drawn from a fixed public benchmark that may already be widely represented online.

The benchmark is designed to isolate the value of non-local context. Our “no-context” baseline is not a zero-information prompt: it still includes the task statement, the failing test context, the local buggy consumer files, and the local test files. What it withholds is surfaced dependency-side context. For dual-repo tasks, this distinction is central: the consumer edit is local, but the correct behavior often lives in provider code or provider documentation. One caveat is that the current query builder uses high-quality task metadata such as `visible_api_names` and `expected_relevant_symbols`, so this benchmark measures retrieval and generation under a well-specified issue query rather than fully raw bug-report understanding.

6.2 Experimental Setup

Our experimental design follows a simple principle: benchmark against a strong baseline, then run controlled ablations to isolate the contribution of each system component. Concretely, our experiments form a matrix over task family, context source, chunking strategy, code-generation model, and embedding model. In our current local run configuration, we sweep four retrieval settings (`grep`, `code`, `wiki`, and `combined`), use base chunking, fix `top_k=5`, and compare six code generators: `bedrock/anthropic.claude-sonnet-4-6`, `bedrock/amazon.nova-pro-v1:0`, `bedrock/deepseek.v3.2`, `bedrock/google.gemma-3-12b-it`, `bedrock/meta.llama4-scout-17b-instruct-v1:0`, and `openai/gpt-5.4`. The current local runs use Titan embeddings (`amazon.titan-embed-text-v2:0`); the evaluation code also supports separate sweeps over alternative embedding models such as `text-embedding-3-large`.

We compare five retrieval conditions. **Baseline** uses only the local prompt and is intentionally strong: it already includes the task statement, failing-test context, local buggy consumer files, and local test files. **Grep** is a lexical baseline that derives up to 12 query terms from the task text and metadata, runs `rg` over non-test Python files, and surfaces snippets from the highest-scoring files; it is included as an IDE-style lexical-search baseline, since many coding assistants still rely heavily on repository text search. **Code** retrieves indexed code chunks from Lighthouse. **Wiki** retrieves generated repository documentation. **Combined** fuses multiple Lighthouse result lists into a single context bundle: code results, wiki results, and, when an AST-oriented auxiliary code index is enabled, results from that alternate chunked code view. This makes the retrieval source itself a first ablation axis, allowing us to ask whether gains come from lexical search, retrieved implementation context, retrieved documentation, or their combination. On the Lighthouse side, code search and wiki search are both hybrid: each combines vector

retrieval with PostgreSQL full-text search and fuses the ranked lists using reciprocal-rank fusion.

Prompting is held constant on the generation side. Both Bedrock and OpenAI models are asked to return only a unified diff patch; the evaluator extracts the patch, applies it, and runs the declared tests. Repair tasks and feature tasks share the same harness, but the feature families are structurally different because they require implementation synthesis against a test specification rather than correction of a single faulty operator or coercion rule.

This synthetic construction is important not only for control, but also for validity. Because the evaluation tasks are instantiated locally rather than copied from a fixed public benchmark, the risk that a model has seen the exact benchmark tests during pretraining is much lower than in evaluations built entirely from long-standing public datasets.

We also include an auxiliary imported ablation focused specifically on retrieval representation. These runs compare an IDE-style `grep` baseline against an AST-oriented retrieval view under repeated trials on the same synthetic families. The imported bundle is narrower than the main experiment matrix because some model cells are incomplete, but it is useful for stress-testing the claim that retrieval quality depends not just on whether context is surfaced, but on how that context is represented and chunked.

6.3 Key Metrics

Our primary metric is *score*, defined as

$$\text{score} = \frac{\# \text{ resolved instances}}{\# \text{ total instances}}.$$

A task is resolved only when the model emits a non-empty patch, the patch applies cleanly, and the declared failing tests pass. We also track intermediate failure modes such as `missing_patch`, `patch_failed`, `unresolved`, and `error`, since these separate localization failures from patch-formatting failures and semantic failures.

To make efficiency interpretable in deployment terms, we convert token counts into dollar cost. For generation, we use

$$C_{\text{gen}} = \frac{T_{\text{in}}}{10^6} p_{\text{in}} + \frac{T_{\text{out}}}{10^6} p_{\text{out}},$$

where T_{in} and T_{out} are the observed input and output tokens, and p_{in} and p_{out} are the provider’s per-million-token prices for the generation model. For embedding traffic, the analogous estimate is

$$C_{\text{emb}} = \frac{T_{\text{emb}}}{10^6} p_{\text{emb}}.$$

We also log wall-clock stage durations for indexing, wiki generation, retrieval, generation, and total runtime. Because indexing and wiki construction are amortizable offline costs,

Table 4: Representative results from completed Titan embedding runs. Generation cost is computed from observed tokens and current provider per-token pricing.

Family	Context	Resolved/10	Score	Gen. cost (\$)	Time (s)
<code>api_contract_mismatch</code>	baseline	1	10.0	0.0110	18.5
<code>api_contract_mismatch</code>	code	8	80.0	0.0185	24.0
<code>api_contract_mismatch</code>	combined	5	50.0	0.0177	42.5
<code>feature_api_integration</code>	wiki	2	20.0	0.0197	40.1
<code>feature_doc_guided</code>	wiki	1	10.0	0.0255	49.4
<code>data_type_coercion</code>	code	9	90.0	0.0121	50.8
<code>data_type_coercion</code>	combined	10	100.0	0.0111	49.7

we treat them separately from per-query retrieval and generation cost.

In addition to score, we report improved, regressed, and unchanged task counts relative to the baseline. This is useful because some retrieval settings improve aggregate score by rescuing many patch-format failures, while others raise score only slightly but change the distribution of failure modes in important ways. One limitation of the current harness is that it evaluates the declared failing pytest slice for each task rather than a full repository regression suite, so the results should be interpreted as targeted fail-to-pass performance on well-scoped coding tasks.

6.4 Results

The results are most useful when read as ablations against the strong baseline rather than as isolated absolute scores. The central question is whether retrieved non-local context improves downstream coding performance enough to justify its added token, latency, and dollar budget, and the completed cells show that the answer depends strongly on which component is added. On the API contract-mismatch family, the `code` ablation is the dominant signal: with Titan embeddings and `top_k=5`, Nova Pro improves from 1/10 resolved tasks in the baseline to 8/10 with `code` retrieval, versus 2/10 with `wiki` and 5/10 with `combined`. The code condition also yields 7 improved tasks and 0 regressions relative to baseline. This is the clearest evidence for Lighthouse’s core claim: when the fix depends on dependency-side semantics, surfacing the provider implementation can change the outcome materially.

The retrieval-source ablation also shows that different views help on different task types. On the documentation-guided feature family, `wiki` is the only Nova retrieval setting that improves over baseline (1/10 versus 0/10 for both `code` and `combined`), while on API feature integration `wiki` ties `code` at 2/10 and both outperform `combined`. This is consistent with the intuition that repository documentation and repository code expose different forms of non-local context: API contract repair depends most on provider implementation details, while documentation-heavy feature work benefits more from surfaced narrative or reference material.

The imported `grep`-versus-AST ablation sharpens the same point from a different angle. For `bedrock/anthropic.claude-sonnet-4-6`, `grep` is already a strong baseline, reaching 100% mean score on API contract mismatch, API feature integration, and type coercion, and 70% on documentation-guided feature work and wrong-operator repair across three repeated trials. Replacing lexical search with the AST-oriented retrieval view is not uniformly better: it drops slightly on API contract mismatch (from 100% to 90%) and type coercion (from 100% to 96.7%), while leaving API feature integration and documentation-behavior repair unchanged at 100% and 90%, respectively. However, it improves documentation-guided feature work from 70% to 90% and wrong-operator repair from 70% to 90%. This auxiliary ablation is useful precisely because it benchmarks against a strong baseline rather than a straw man: representation choice matters, but its value is family-dependent.

The `combined` setting is therefore best understood as a component ablation over context fusion rather than a universally dominant configuration. In the two completed API contract-mismatch sweeps, `combined` resolves 13/20 instances in aggregate, slightly above `code` at 12/20, which is at least consistent with partial complementarity between code and documentation views. We also observe slices where this advantage appears directly at the single-family level: on `data.type.coercion` with `bedrock/deepseek.v3.2`, `combined` improves from 9/10 resolved tasks with `code` to 10/10, while slightly reducing estimated generation cost and runtime. At the same time, the family-level Nova results show that broader context is not automatically better: on some slices, especially when the added material is noisy for the task, `combined` pays extra latency without delivering the best score.

Cost makes these ablations sharper. On the Nova API contract-mismatch run, generation cost rises from about \$0.0110 in the baseline to \$0.0185 with the `code` ablation, but that additional spend buys a jump from 10% to 80% resolved. The code-generation-model ablation reveals a complementary trade-off. On the same family, `bedrock/google.gemma-3-12b-it` with `combined` retrieval also reaches 8/10 resolved tasks, but at an estimated generation cost of roughly \$0.0020, about $9.4\times$ cheaper than Nova Pro with `code`. The drawback is stability: Gemma is much less reliable across the remaining families, and its latency on this slice is substantially worse (110.2s versus 24.0s). The broader interpretation is that retrieval can narrow the quality gap for cheaper generators, but the relevant trade-off is not only accuracy versus cost; it is accuracy versus cost versus latency.

We observe a similar pattern in the embedding-model ablation. Swapping the Bedrock embedding model (`amazon.titan-embed-text-v2:0`) for OpenAI `text-embedding-3-large` produces qualitatively similar retrieval behavior, so the main practical distinction is cost rather than score. Titan Text Embeddings V2 is priced at \$0.02 per 1M input tokens in the Bedrock documentation, while `text-embedding-3-large` is priced at \$0.13 per 1M tokens in OpenAI’s current model documentation, making the OpenAI option roughly $6.5\times$ more expensive per embedding token. Given the small observed accuracy difference, this ablation shifts the cost-performance frontier toward Titan for routine indexing and retrieval.

7 Future Work and Next Steps

Lighthouse is a research instrument and proof of concept with a clear path toward becoming a production product. Future work spans two lenses: product maturity and research depth.

On the product side, the clearest next step is automated webhook setup so that repositories re-index on every push without manual intervention, removing the largest remaining onboarding friction point. We also want incremental wiki generation so only pages affected by changed source chunks are regenerated, reducing the dominant ingestion cost, and expanded search coverage over agent history, commits and PRs, and issue threads so Lighthouse can retrieve more of the intent and debugging context that lives outside the code itself.

On the research side, we see promise in graph-structured indexing that encodes dependency relationships directly, enabling retrieval over code structure as well as semantics, and in agentic retrieval that iteratively refines queries after inspecting early results. We also want to study top- k tuning and budget-aware retrieval so the system can adapt result count to the task and available context window, and to expand indexing beyond source code to external documentation and dependency metadata, which would improve support for third-party APIs and version-specific behavior.

8 Team Reflection

8.1 Infrastructure as a First-Class Concern

The infrastructure lecture’s layered view of ML systems became very concrete in Lighthouse. We quickly learned that retrieval quality depended on having reliable foundations first: durable workflow orchestration, stable storage, and a zero-downtime indexing process mattered more than any ranking tweak we wanted to try. That pushed us toward a build-vs-buy split similar to the course framing: we relied on managed infrastructure such as ECS, RDS, S3, and Milvus, while building the retrieval logic and agent-facing interfaces ourselves. Writing Terraform early and containerizing services with Docker also made deployment and environment parity much more manageable.

8.2 AI-Assisted Development and the Value of Tests

Using AI coding agents throughout the project gave us a direct view of both their strengths and their risks. For scoped implementation tasks they were fast and often useful, but they were also prone to violating local conventions or hidden invariants. The most effective counterweight was testing: the assignment pushed us to build a large, high-coverage suite, and that suite became our main mechanism for supervising agent-generated code. In practice, writing tests first gave us a clean contract to hand to an agent and a reliable way to catch mismatches between its assumptions and ours.

8.3 Production Hardness and the Cost of Experimentation

Building Lighthouse as something production-capable exposed problems that would not appear in a simple prototype. Our indexing workflow, for example, needed to avoid serving partially updated results during re-indexing, which led to a publish-and-switch design and much more careful coordination between services. We also learned that experimentation has real operational cost: every retrieval variant affects embedding runs, reranking, and LLM-based evaluation spend. That forced us to be disciplined about isolating variables and choosing experiments that were worth the cost.

8.4 Abstractions, Collaboration, and Configurability

Because Lighthouse spans multiple services, abstraction boundaries ended up mattering as much as the retrieval algorithms themselves. We needed enough configurability to swap chunking, ranking, and embedding strategies for experiments, but not so many exposed knobs that each service became difficult to understand. The compromise was to keep configuration explicit at service boundaries and keep internals relatively opinionated. That structure also helped with collaboration and code review, especially when integrating AI-generated code that was often locally plausible but architecturally inconsistent.

8.5 Research and Product in the Same System

The most important framing decision we made was to treat Lighthouse as both a research artifact and a production system. The research side gave us concrete evaluation questions about retrieval quality and downstream agent performance, while the product side forced us to care about reliability, observability, and deployability. In practice, these goals reinforced each other: stronger infrastructure made experimentation easier, and clearer evaluation made engineering tradeoffs easier to justify. That combination made the project more demanding, but it also made the result feel substantially more credible than a prototype built only for a demo.

References

- [1] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [2] Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of EMNLP 2020*, 6769–6781.
- [3] Husain, H., et al. (2019). CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *arXiv preprint arXiv:1909.09436*.
- [4] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *Proceedings of SIGIR 2009*, 758–759.
- [5] Wang, Z., et al. (2024). CodeRAG-Bench: Can Retrieval Augment Code Generation? *arXiv preprint arXiv:2406.14497*.
- [6] Liu, T., et al. (2023). RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. *arXiv preprint arXiv:2306.03091*.
- [7] Zan, D., et al. (2024). GitChameleon: Unmasking the Version-Conditioned Code Generation Challenge. *arXiv preprint arXiv:2411.05830*.